LIGHTWEIGHT CRYPTOGRAPHY IN IoT…….A STUDY OF ASCON-128

1. INTRODUCTION

Cryptography consumes power, memory and takes time, IOT devices are used as an essential part of human life and something like that needs to work immediately with less resources consumption, that's where lightweight Cryptography comes in.

Lightweight Cryptography is a specialized subfield of cryptography designed to provide security for devices with minimal resources. The rise of the Internet of Things (IoT), wireless sensors, access controls, smart homes and smart devices has created a growing need for secure communication in devices that have limited resources (Stallings, 2020). Common encryption methods like AES, RSA, and SHA were built for systems that have enough processing power, memory, and energy to run them. But many devices used today, such as RFID tags, wearables, medical sensors, and access control systems, do not have that kind of capacity. These devices work with very little processing power, storage, battery, and network bandwidth (Poschmann, 2009). Because of this lightweight cryptography was developed, which is a type of cryptography designed to keep devices secure without using too many resources.

Lightweight cryptography simply means encryption, decryption and hashing methods that are built to work on small, low-power devices. The goal is to find a balance between security, speed, and cost of use (Hatzivasilis, Floros and Fysarakis, 2018). Unlike regular cryptography, lightweight algorithms are designed to use less memory, less energy, and less processing power, while still keeping data safe. This makes them a good fit for IoT systems.

Over time, lightweight cryptography has gained more attention because cyberattacks on connected devices have been increasing. Many low-cost devices cannot run standard encryption properly, so they are often left with weak security, which makes them easy targets for attacks like data theft, device impersonation, and unauthorized access (Stallings, 2020). Lightweight cryptographic algorithms help fix this problem by making it possible to protect data and verify identity even on devices with limited capabilities.

Over the past twenty years, many lightweight block ciphers, stream ciphers, and hash functions have been developed. Some examples include PRESENT, CLEFIA, SIMON, SPECK, LED, and ASCON (Bogdanov et al., 2007; Beaulieu et al., 2015). Out of these, ASCON became the most recognized after the National Institute of Standards and Technology (NIST) chose it as the official standard for lightweight cryptography in 2023 (NIST, 2023). This showed just how important efficient and secure encryption has become, especially as IoT continues to grow.

However, lightweight cryptography still comes with its own challenges. Developers have to be careful when simplifying encryption, because making it too simple can leave it open to attacks (Dobraunig et al., 2021). Also, because IoT devices vary so much in terms of design and purpose, it is hard to create one solution that works for all of them. This is why research in this area is still ongoing, with focus on improving security, saving energy, and protecting against attacks.

This research focuses on ASCON-128(AEAD) an algorithm from the Ascon Family, looking at how it works and how well it performs. ASCON-128 is a lightweight encryption algorithm that is designed to protect data by keeping it private and making sure it is not tampered with, all while using very little processing power and energy (Dobraunig et al., 2021). It is built on a design structure called sponge construction, which helps it run efficiently on devices with limited hardware and software capacity. Because of its strong security, simplicity, and good performance, ASCON-128 is part of the Ascon Family that was chosen by the National Institute of Standards and Technology as the official lightweight cryptography standard in 2023 (NIST, 2023). Since it works well on IoT devices and low-power devices, it is a great choice for testing how effective lightweight encryption can be in real-world situations.

2.0 LITERATURE REVIEW

2.1 Introduction to Lightweight Cryptography

Lightweight cryptography has grown into an important area of study, mainly because of how fast the Internet of Things (IoT) has expanded. Devices like wireless sensor networks, RFID systems, embedded devices, and access control systems are now everywhere. The problem is that most of these devices have very limited processing power, small memory and low energy, Older cryptographic algorithms like AES, RSA, and SHA were built for regular computers that have enough resources to run them, so they do not work well on these smaller devices (Stallings, 2020).

For a long time, AES was the standard for symmetric encryption. But Bogdanov et al. (2007) found that AES-128 needs around 3,000–4,000 Gate Equivalents (GE) to run, which is too much for very small devices like passive RFID tags that can only spare about 2,000 GE for everything they do. (Rolfes et al., 2008) This gap pushed researchers to start developing lighter cryptographic algorithms that could work with limited resources.

The aim of lightweight cryptography is to offer reasonable security without demanding too much from the hardware. It tries to keep things small in terms of processing time, memory use, and energy. According to Thakor, Razzaque and Khandaker (2020), the main challenge is finding the right balance, because making an algorithm simpler can sometimes make it easier for attackers to break.

2.2 Evolution of Lightweight Cryptographic Primitives

2.2.1 Lightweight Block Ciphers

In the early days of lightweight cryptography, most researchers were focused on making block ciphers small enough to work on constrained hardware and embedded systems.

PRESENT (2007)

One of the first and most well-known lightweight block ciphers was PRESENT, introduced by Bogdanov et al. (2007). PRESENT works with a 64-bit block size and supports either 80-bit or 128-bit keys. Instead of the usual 8-bit S-boxes, it uses smaller 4-bit S-boxes, which helps reduce hardware requirements. The whole cipher only needed about 1,570 GE, which made it a good fit for passive RFID tags and very low-cost devices. (Lim & Korkishko, 2006) However, as security needs grew over time, its 64-bit block size and the older 80-bit key option started to show their limitations.

SIMON and SPECK (2013)

In 2013, the National Security Agency (NSA) introduced two new lightweight block ciphers called SIMON and SPECK (Beaulieu et al., 2015). Both ciphers use ARX operations, which stands for Addition, Rotation, and XOR. These operations are simple and cheap to compute, making the ciphers well-suited for constrained devices.

SIMON was designed more for hardware use and needed roughly 1,500 GE.

SPECK was built for software environments and could run quickly with a code size of only about 1–2 KB on 8-bit microcontrollers.

Even though both ciphers performed well, NIST never officially standardized them, I can remember Prof Bill Buchanan saying the NSA allegedly put a backdoor in the ciphers which limited how widely they were used in commercial security products.

GIFT (2017)

GIFT came after PRESENT and was designed to improve on it following the same prototype, It offered better security and stronger resistance against attacks while still keeping the hardware cost low. GIFT supports both 64-bit and 128-bit block sizes and fits within about 1,500 GE. (Efficient Implementation of PRESENT and GIFT on Quantum Computers, 2020) It became popular for use in smart cards, industrial sensors, and secure embedded systems because of its low power use and its ability to resist differential cryptanalysis.

2.2.2 Shift Toward Authenticated Encryption (AEAD)

Over time, researchers began moving away from just encryption and toward something called Authenticated Encryption with Associated Data (AEAD), because it combines confidentiality, integrity and authenticity in a single primitive. Schemes such as GIFT-COFB, Elephant, ISAP, Grain-128AEAD and Ascon have been evaluated extensively for their suitability in resource limited environments (Buchanan, 2023), The reason for this shift was simple, encrypting data alone was not enough to keep IoT communication safe. Attackers could still tamper with the ciphertext or change protocol information without being detected.

AEAD solves this by doing two things at once:

-Confidentiality (it encrypts the actual data)

-Integrity and authentication (it checks whether the data has been tampered with)

This became important in IoT systems where extra information like routing details, packet headers, sensor IDs, and timestamps also needs to be protected, even when it is not encrypted.

The CAESAR competition and later the NIST Lightweight Cryptography project helped push this shift forward by encouraging algorithms that could handle both encryption and authentication in one lightweight package. Today, AEAD is seen as a basic requirement for safe communication between constrained devices.

2.3 The NIST Lightweight Cryptography Standardization Process

A major step in the development of lightweight cryptography was the launch of the NIST Lightweight Cryptography Standardization Project. The project started in 2018 and reviewed 57 algorithms submitted by researchers from around the world. (Lightweight Cryptography (LWC) Standardization: Round 1 Candidates Announced, 2019)

The evaluation looked at several factors like Security strength, Hardware effectiveness, Software performance, Energy consumption and Resistance to attacks and flexibility.

Studies on the finalists, which included ASCON, GIFT-COFB, and TinyJAMBU among others, showed that sponge-based and permutation-based designs were becoming more preferred over older approaches like Feistel or substitution-permutation networks. (Turan et al., n.d.)

After rounds of review, NIST officially chose ASCON as the winner in February 2023, based on its strong security, flexibility across different devices, and good resistance against side-channel attacks (NIST, 2023).

Name	Type	Variant	Underlying Primitive	State (Bits)	Key (Bits)	Mode	Rate/Block (Bits)	Tag (Bits)	Security (Bits)
Ascon	Sponge	Ascon-128	Ascon-p	320	128	Duplex	64	128	128
		Ascon-128a	Ascon-p	320	128	Duplex	128	128	128
Elephant	Sponge	Jumbo	Spongent	176	128	Elephant	176	64	127
		Dumbo	Spongent	160	128	Elephant	160	64	112
		Delirium	Keccak	200	128	Elephant	176	128	127
GIFT-COFB	Block	GIFT-COFB	GIFT-128	192	128	COFB	128	128	128
Grain-128AEAD	Stream	Grain-128AEAD	N/A	256	128	N/A	1	64	128
ISAP	Sponge	ISAP-A-128	Ascon-p	320	128	ISAP	64	128	128
		ISAP-K-128	Keccak	400	128	ISAP	144	128	128
		ISAP-K-128A	Keccak	400	128	ISAP	144	128	128
		ISAP-A-128A	Ascon-p	320	128	ISAP	64	128	128
PHOTON-Beetle	Sponge	PHOTON-Beetle-AEAD[128]	PHOTON256	256	128	Beetle	128	256	121
		PHOTON-Beetle-AEAD	PHOTON256	256	128	Beetle	32	256	128
Romulus	Block	Romulus-M	Skinny-128-384	384	128	COFB	128	128	128
		Romulus-N	Skinny-128-384	384	128	COFB	128	128	128
		Romulus-T	Skinny-128-384	384	128	COFB	128	128	128
SPARKLE	Sponge	SCHWAEMM256-128	SPARKLE	384	128	SPARKLE	256	128	120
		SCHWAEMM128-128	SPARKLE	256	128	SPARKLE	128	128	120
		SCHWAEMM192-192	SPARKLE	384	192	SPARKLE	192	192	184
		SCHWAEMM256-256	SPARKLE	512	256	SPARKLE	256	256	248
TinyJambu	Sponge	TinyJambu	TinyJambu	128	128	TinyJambu	32	64	120
Xoodyak	Sponge	Xoodyak	Xoodoo	384	128	Cyclist	352	128	128

Figure 1: Specifications of the NIST LWC finalist algorithms (Buchanan 2023)

NIST later published the final lightweight cryptography standard as SP 800-232 on August 13, 2025, under the title Ascon-Based Lightweight Cryptography Standards for Constrained Devices. The standard defined four main ASCON-based functions:

-Ascon-AEAD128

-Ascon-Hash256

-Ascon-XOF128

-Ascon-CXOF128

This family-based approach is useful in IoT development because it allows developers to reuse the same internal structure across multiple cryptographic tasks. This reduces both the complexity of the software and the amount of hardware space needed. (Gookyi & Ryoo, 2022)

2.4 The ASCON Family

ASCON is a permutation-based sponge-construction cryptographic suite designed by Dobraunig et al. specifically for lightweight use. It was built to work on 8-bit microcontrollers and low-power hardware while still offering strong and up-to-date security.

The ASCON family has four standardized versions:

i. ASCON-128 – The main AEAD algorithm, using a 128-bit key, 128-bit nonce, and 128-bit authentication tag

ii. ASCON-128a – A faster AEAD version made for IoT applications that handle larger amounts of associated data

iii. ASCON-HASH – A lightweight 256-bit hash function for data integrity plus message authentication.

iv. ASCON-XOF – An extendable output function that can produce variable-length outputs, useful for hashing and lightweight signature tasks.

ASCON uses a 320-bit internal state, split into: 128-bit capacity & 192-bit rate

Its main permutation runs 12 rounds, split into an initialization phase and a finalization phase. (Turan et al., n.d.) Each round involves:

-Adding round constants

-A substitution layer using compact 5-bit S-boxes

-A linear diffusion layer using bitwise rotations and XOR operations

This design allows ASCON to be implemented in about 2,500 GE on hardware and only needs around 2-3 KB of code on 8-bit AVR microcontrollers (Dobraunig et al., 2021).

2.5 Why ASCON Was Selected by NIST

ASCON stood out from the other candidates during the NIST evaluation for a number of clear reasons.

i. Flexibility: ASCON can handle authenticated encryption, hashing, and extendable output function using the same internal permutation. This means developers do not need to implement separate algorithms for each task, which made things easy on resource limited systems.

ii. Software Efficiency: While some lightweight algorithms only perform well on hardware, ASCON works efficiently on both 32-bit and 64-bit processors and still runs well on 8-bit microcontrollers.

iii. Side-Channel Resistance: ASCON's permutation-based structure makes it easier to apply masking, which is a technique used to hide power consumption patterns during encryption. This makes it harder for attackers to use side-channel methods to break the cipher, which is a real concern in IoT hardware.

iv. Balanced Security and Efficiency: Studies by Dobraunig et al. (2021) and others showed that ASCON can deliver strong 128-bit security while still using low energy, taking up little hardware space, and maintaining good throughput.

2.6 Importance of AEAD in IoT Security

In modern IoT environments, simple encryption is often not enough. Attackers can still interfere with things like packet headers, routing details, timestamps, or sensor metadata even when the main data is encrypted.

AEAD tackles this problem directly by protecting both the encrypted payload (the actual data) and the associated unencrypted metadata (extra information sent alongside the data)

NIST's SP 800-232 specifically included Ascon-AEAD128 as the main lightweight authenticated encryption method for resource limited devices. This shows how important it has become to protect both data content and data context together.

AEAD is important in areas such as:

-Smart healthcare devices

-Industrial IoT systems

-Wireless sensor networks

-Access controls systems

-RFID authentication

Since these systems frequently send sensitive information over wireless networks that are not always secure, having both encryption and authentication working together is essential. Without it, systems remain vulnerable to attacks like spoofing, replay attacks and unauthorized data modification.

3.0 IMPLEMENTATION

I am using a python code for the implementation of ASCON-128, the code was gotten from Bill Buchanan (2023) “Slow and Steady wins the lightweight race: Meet ASCON”, which he got from the Python3 implementation of Ascon on GitHub by Maria Eichlseder and Marcel Nageler.

I added an evaluation suite to the code with metrics such as Tamper test, AD binding, Encryption and Decryption timing, Throughput, Memory usage and Security Avalanche effect.

The code support Ascon-128, 128a & 80Pq, but I would be using the Ascon-128 for this work. based on reasons for page capping, I will only be posting snippet of the python code for the encryption & decryption, Ascon permutation & finalization, then finally the code output. But the full code will be on my GitHub.

```python
def ascon_encrypt(key, nonce, associateddata, plaintext, variant="Ascon-128"):
    """
    returns ciphertext||tag (tag is 16 bytes)
    """
    assert variant in ["Ascon-128", "Ascon-128a", "Ascon-80pq"]
    if variant in ["Ascon-128", "Ascon-128a"]:
        assert len(key) == 16 and len(nonce) == 16
    if variant == "Ascon-80pq":
        assert len(key) == 20 and len(nonce) == 16

    S = [0, 0, 0, 0, 0]
    k = len(key) * 8
    a = 12
    b = 8 if variant == "Ascon-128a" else 6
    rate = 16 if variant == "Ascon-128a" else 8

    ascon_initialize(S, k, rate, a, b, key, nonce)
    ascon_process_associated_data(S, b, rate, associateddata)
    ciphertext = ascon_process_plaintext(S, b, rate, plaintext)
    tag = ascon_finalize(S, rate, a, key)
    return ciphertext + tag


def ascon_decrypt(key, nonce, associateddata, ciphertext_and_tag, variant="Ascon-
128"):
    """
    returns plaintext bytes or None if verification fails
    """
    assert variant in ["Ascon-128", "Ascon-128a", "Ascon-80pq"]
    if variant in ["Ascon-128", "Ascon-128a"]:
        assert len(key) == 16 and len(nonce) == 16 and len(ciphertext_and_tag) >=
16
    if variant == "Ascon-80pq":
        assert len(key) == 20 and len(nonce) == 16 and len(ciphertext_and_tag) >=
16

    ct = ciphertext_and_tag[:-16]
    tag = ciphertext_and_tag[-16:]

    S = [0, 0, 0, 0, 0]
    k = len(key) * 8
    a = 12
    b = 8 if variant == "Ascon-128a" else 6
```

```
    rate = 16 if variant == "Ascon-128a" else 8

    ascon_initialize(S, k, rate, a, b, key, nonce)
    ascon_process_associated_data(S, b, rate, associateddata)
    plaintext = ascon_process_ciphertext(S, b, rate, ct)
    comp_tag = ascon_finalize(S, rate, a, key)

    # constant-time tag check
    if hmac.compare_digest(comp_tag, tag):
        return plaintext
    return None


def ascon_initialize(S, k, rate, a, b, key, nonce):
    iv_zero_key_nonce = to_bytes([k, rate * 8, a, b] + (20 - len(key)) * [0]) +
key + nonce
    S[0], S[1], S[2], S[3], S[4] = bytes_to_state(iv_zero_key_nonce)

    ascon_permutation(S, a)

    zero_key = bytes_to_state(zero_bytes(40 - len(key)) + key)
    S[0] ^= zero_key[0]
    S[1] ^= zero_key[1]
    S[2] ^= zero_key[2]
    S[3] ^= zero_key[3]
    S[4] ^= zero_key[4]


def ascon_process_associated_data(S, b, rate, associateddata):
    if len(associateddata) > 0:
        a_zeros = rate - (len(associateddata) % rate) - 1
        a_padding = to_bytes([0x80] + [0 for _ in range(a_zeros)])
        a_padded = associateddata + a_padding

        for block in range(0, len(a_padded), rate):
            S[0] ^= bytes_to_int(a_padded[block:block + 8])
            if rate == 16:
                S[1] ^= bytes_to_int(a_padded[block + 8:block + 16])
            ascon_permutation(S, b)

    # domain separation
    S[4] ^= 1

def ascon_finalize(S, rate, a, key):
    assert len(key) in [16, 20]
```

```
    S[rate // 8 + 0] ^= bytes_to_int(key[0:8])
    S[rate // 8 + 1] ^= bytes_to_int(key[8:16])
    S[rate // 8 + 2] ^= bytes_to_int(key[16:] + zero_bytes(24 - len(key)))

    ascon_permutation(S, a)

    S[3] ^= bytes_to_int(key[-16:-8])
    S[4] ^= bytes_to_int(key[-8:])
    return int_to_bytes(S[3], 8) + int_to_bytes(S[4], 8)


def ascon_permutation(S, rounds=1):
    assert rounds <= 12
    for r in range(12 - rounds, 12):
        # add round constants
        S[2] ^= (0xF0 - r * 0x10 + r * 0x01)

        # substitution layer
        S[0] ^= S[4]
        S[4] ^= S[3]
        S[2] ^= S[1]
        T = [((S[i] ^ 0xFFFFFFFFFFFFFFFF) & S[(i + 1) % 5]) for i in range(5)]
        for i in range(5):
            S[i] ^= T[(i + 1) % 5]
        S[1] ^= S[0]
        S[0] ^= S[4]
        S[3] ^= S[2]
        S[2] ^= 0xFFFFFFFFFFFFFFFF

        # linear diffusion layer
        S[0] ^= rotr(S[0], 19) ^ rotr(S[0], 28)
        S[1] ^= rotr(S[1], 61) ^ rotr(S[1], 39)
        S[2] ^= rotr(S[2],  1) ^ rotr(S[2],  6)
        S[3] ^= rotr(S[3], 10) ^ rotr(S[3], 17)
        S[4] ^= rotr(S[4],  7) ^ rotr(S[4], 41)
```

This code is a complete software implementation of the ASCON-128 encryption algorithm, built using the Python programming language. It was created to show how ASCON-128 works and how well it performs in devices with limited resources.

The code is broken down into several key steps of the ASCON encryption process. These steps include initialization, processing associated data, encrypting a message, decrypting the message, finishing the process, and checking the authentication tag. The code follows the sponge-based

design described in the ASCON specification, where the bits are repeatedly transformed through permutation to protect both the privacy and the integrity of the data.

The **ascon_encrypt()** function handles the full encryption process. It sets up the internal state using a secret key and a nonce, takes in the associated data, encrypts the message, and produces a 128-bit authentication tag.

The **ascon_decrypt()** function does the opposite by decrypting the message and checking the authentication tag using a function called **hmac.compare_digest(),** which helps prevent timing attacks.

Another important part of the code is the **ascon_permutation()** function, which is the core of ASCON. This function runs several rounds of transformation that involve adding round constants, swapping values using S-box operations, and mixing data using bitwise rotation and XOR operations. Together, these steps make the encryption strong and secure.

The code also includes an evaluation section that checks how well the algorithm performs and how secure it is. Overall, the results show that ASCON-128 is able to encrypt data efficiently while using very little memory, keeping the design simple, and maintaining strong security. This makes it a very good option for lightweight encryption in IoT.

And a sample run is:

```
PS C:\Users\aq> & C:/Users/aq/AppData/Roaming/uv/python/cpython-3.14.4-windows-x86_64-none/python.exe "c:/Users
/aq/my own code.py"
============================================
      ASCON AEAD EVALUATION
============================================

[1] Validation Check (Encrypt/Decrypt + Tag Verify + AD binding)
    Variant:       Ascon-128
    Plaintext:     b'MY NAME IS MOHAMMED'
    AD (hex):      50524f46462042494c4c204953205448452042455354204c45435455524552
    CT||TAG (hex): a4b030209e4c9c7646fd8860d71b135c7aff3be14c6a106ca6500ea326f0a4901ca1dd
    Decrypted:     b'MY NAME IS MOHAMMED'
    Status:        SUCCESS

    Tamper test:    PASS (rejected)
    AD binding:     PASS (rejected)

[2] Throughput & Timing (Encryption)
    Messages:        2000
    Payload bytes:   19
    Total Time:      6.9730 seconds
    Avg Latency:     3.487 ms/message
    Throughput:      5.32 KB/s

[2b] Throughput & Timing (Decryption + Tag Verify)
    Messages:        2000
    Total Time:      6.7740 seconds
    Avg Latency:     3.387 ms/message

[3] Memory Consumption
    Peak RAM usage:   0.84 KB

[4] Security: Avalanche Effect (first 64 bits of ciphertext)
    Bits changed:     1 / 64
    Avalanche %:      1.56% (Ideal ~50%)
============================================
```

Figure 2: Output of the above Ascon AEAD code

4.0 EVALUATION

The output showed the successful implementation of the ASCON-128 AEAD algorithm, this evaluation focuses on the correctness, efficiency, memory consumption, security and performance (test loop).

4.1 Validation and Correctness

The results confirmed the encryption and decryption process operated successfully as the plaintext message "**MY NAME IS MOHAMMED**" was encrypted using ASCON-128 together with associated data, which result in producing a ciphertext and authentication tag.

During decryption, the original plaintext was recovered correctly and the output status displayed "**SUCCESS**", meaning the procedures were performed accurately.

4.2 Security Evaluation and analysis: Tamper Test and Avalanche Effect

-Tamper test: The tamper test successfully rejected modified ciphertext, while the associated data binding test rejected altered data, protecting both confidentiality and integrity.

-Avalanche Effect: this test evaluated the algorithm by modifying a single bit in the plaintext and measuring the resulting changes in the ciphertext.

The evaluation produced:

- Bits changed: 1 out of 64

- Avalanche percentage: 1.56%

In ideal cryptographic systems, the effect should approach approximately 50% where a small change in input produces significant change, however the code was appear faulty as I manually changed the bit and it achieved the 50% effect.

```
Plaintext:     b'MY NAME IS MOHAMMAD'
AD (hex):      50524f46462042494c4c204953205448452042455354204c45435455524552
CT||TAG (hex): a4b030209e4c9c7646fd8860d71b135c7afb3b3f1d6227db0abad63d8fb5f5b0665c78
Decrypted:     b'MY NAME IS MOHAMMAD'
```

Figure.3:

```
Plaintext:     b'MY NAME IS MOHAMMED'
AD (hex):      50524f46462042494c4c204953205448452042455354204c45435455524552
CT||TAG (hex): a4b030209e4c9c7646fd8860d71b135c7aff3be14c6a106ca6500ea326f0a4901ca1dd
Decrypted:     b'MY NAME IS MOHAMMED'
```

Figure.4: This shows up to 50% of the cipher text being changed after flipping the letter “A” ”E” in the name, this Indicate strong diffusion and resistance to cryptanalysis.

4.3 Performance Evaluation

The throughput and timing analysis measured the computational efficiency of the implementation of over 2,000 encryption and decryption operations, the results from figure.2 indicate that the algorithm achieves low latency while maintaining lightweight computational requirements, this demonstrate that the algorithm can process data In resource limited environments such as IoT.

4.4 Memory Consumption

The analysis showed a peak RAM usage of only “**0.84 KB**” showing the lightweight nature of the implementation, this extremely low memory usage confirms the suitability of ASCON-128 for

resource limited platforms (IoT), as low memory is one of the primary advantage of lightweight cryptographic algorithms compared to traditional encryption standards.

4.5 Performance Experiment (Test Loop)

An automated loop was executed to measure the latency and throughput of the algorithm across various payload sizes, the code is on my github titled as “test loop”

```
def run_automated_test_loop():
    variants = ["Ascon-128"]
    test_sizes = [0, 1, 7, 8, 9, 15, 16, 17, 32, 128] # Testing different block boundaries
```

Figure.6:

And a sample run is:

```
================================================================================
                          ASCON AUTOMATED TEST LOOP
================================================================================
Variant         | Size    | Enc/Dec  | Integrity  | Throughput
--------------------------------------------------------------------------------
Ascon-128       |    0 B  | PASS     | SECURE     |       0.00 KB/s
Ascon-128       |    1 B  | PASS     | SECURE     |       1.19 KB/s
Ascon-128       |    7 B  | PASS     | SECURE     |       6.77 KB/s
Ascon-128       |    8 B  | PASS     | SECURE     |       8.15 KB/s
Ascon-128       |    9 B  | PASS     | SECURE     |       9.91 KB/s
Ascon-128       |   15 B  | PASS     | SECURE     |      17.08 KB/s
Ascon-128       |   16 B  | PASS     | SECURE     |       7.34 KB/s
Ascon-128       |   17 B  | PASS     | SECURE     |      14.56 KB/s
Ascon-128       |   32 B  | PASS     | SECURE     |      19.07 KB/s
Ascon-128       |  128 B  | PASS     | SECURE     |      46.13 KB/s
================================================================================
TEST LOOP COMPLETE.
```

Figure.7: test loop output of figure.6

The throughput increases with larger payloads because the fixed “initialization” and “Finalization” overhead is spread out over more data blocks, this confirms Ascons efficiency for streaming IoT data.

5.0 CONCLUSIONS

This research examines lightweight cryptography with focus on the implementation and evaluation of the ASCON-128 AEAD algorithm. The study explored the evolution of lightweight cryptography and the limitations of traditional encryption algorithms in resource limited environment.

Performance evaluation showed that the implementation achieved low memory consumption with a peak RAM usage of 0.84KB, confirming the lightweight characteristics of ASCON-128, timing and throughput measurements also demonstrated efficient execution suitable for IoT where computational resources, memory and energy are limited. The validation results confirmed the encryption and decryption process operated successfully as the plaintext message was encrypted using ASCON-128 together with associated data, which result in producing a ciphertext and authentication tag.

Security analysis through avalanche effect testing showed the sensitivity of the algorithm to modifications, even though the code didn't give the intended result, I manualy showed how I got an intended output.

Overall, this research confirms that ASCON-128 AEAD provides an effective lightweight cryptographic solution capable of balancing security, efficiency and low resource consumption. The sponge-based permutation, resistance to being tampered and it standardization by NIST make it highly suitable for modern IoT.

FUTURE WORK

Several areas may be explored in future research to further improve and extend this work, the research should investigate the **Side-Channel Resistance** of this python implementation on actual hardware (e.g, Raspberry Pi) to measure power-leakage during the permutation phase. Additionally, the integration of **Post-Quantum Cryptography (PQC)** signatures with lightweight ciphers should be explored as the next frontier in IoT.

REFERENCES

Bill Buchanan (2023) "Slow and Steady wins the lightweight race: Meet ASCON"

Stallings, W. (2020) *Cryptography and Network Security: Principles and Practice*. 8th edn.

NIST (2023) *NIST Selects ASCON as the Lightweight Cryptography Standard*. Available at: NIST Lightweight Cryptography Project

Poschmann, A. (2009) *Lightweight Cryptography: Cryptographic Engineering for a Pervasive World*. PhD thesis. Ruhr University Bochum.

Hatzivasilis, M., Floros, G. and Fysarakis, K. (2018) 'Review of lightweight block ciphers', *Journal of Cryptographic Engineering*, 8(2),

Bogdanov, A., Knudsen, L.R., Leander, G., Paar, C., Poschmann, A., Robshaw, M.J.B., Seurin, Y. and Vikkelsoe, C. (2007) 'PRESENT: An ultra-lightweight block cipher', *Cryptographic Hardware and Embedded Systems – CHES 2007*.

Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B. and Wingers, L. (2015) 'The SIMON and SPECK lightweight block ciphers', *Proceedings of the 52nd Annual Design Automation Conference*.

NIST (2023) *Lightweight Cryptography Standardization Process*. Available at: NIST Official Website.

Dobraunig, C., Eichlseder, M., Mendel, F. and Schläffer, M. (2021) 'ASCON v1.2: Lightweight authenticated encryption and hashing', *Journal of Cryptology*,

Thakor, V.A., Razzaque, M.A. and Khandaker, M.R.A. (2020) 'Lightweight cryptography for IoT: A state-of-the-art'

Rolfes, C., Poschmann, A., Leander, G. and Paar, C. (2008) 'Ultra-lightweight implementations for smart devices -Security for 1000 gate equivalents', *Smart Card Research and Advanced Applications*, Lecture Notes in Computer Science, 5189, pp. 89-103.

McKay, K., Bassham, L., Sonmez Turan, M. and Mouha, N. (2017) *Report on Lightweight Cryptography*. NIST Interagency/Internal Report (NISTIR) 8114.

Gookyi, D.A.N. and Ryoo, K. (2022) 'A lightweight system-on-chip based cryptographic core for low-cost devices', *Sensors*, 22(8), p. 3004.

Jang, K., Song, G., Kim, H., Kwon, H., Kim, H. and Seo, H. (2021) 'Efficient implementation of PRESENT and GIFT on quantum computers', *Applied Sciences*, 11(11), p. 4776.

Lim, C.H. and Korkishko, T. (2006) 'mCrypton -A lightweight block cipher for security of low-cost RFID tags and sensors', Information Security Applications, Lecture Notes in Computer Science, 3786, pp. 243–258.